Online on xv6 System Call

Section: B1 — **Time:** 35 minutes

In this assignment, you will rate limit a system call. A process nominates one system call and a minimum gap in kernel ticks, and from then on the kernel refuses any call of that system call that arrives too soon after the previous one. The kernel also keeps a system wide tally of how many calls it has let through and how many it has refused, which a user space program fetches using kernel space memory copying.

Required System Calls:

1. `throttleSyscall(int syscall_num, int min_ticks)` : From this point on, the calling process may make system call syscall_num **at most once every min_ticks ticks.** A call that arrives sooner must return -1 and the system call must not be carried out at all. Returns 0 on success and -1 if any argument is invalid.
2. `getThrottleStats(struct throttle_stats *ts)` : Copies the current statistics into the provided user space struct pointer. Returns 0 on success and -1 on invalid memory address.

The structure definition is as follows:

```
struct throttle_stats {
    int allowed;   // calls the rate limiter let through
    int denied;    // calls the rate limiter refused for arriving too soon
};
```

Your implementation must satisfy all of the following:

- A process rate limits exactly one system call at a time. Calling `throttleSyscall` again replaces the previous limit.
- The first call made after `throttleSyscall` is always allowed.
- The gap is measured **from the last call** that was allowed. A call that is refused does not restart the clock.
- Only the nominated system call is affected. Every other system call the process makes runs normally.
- A child created by `fork()` **inherits** its parent's limit, but starts with a fresh clock: the child's own first call to that system call is allowed.
- The two counters are shared by the whole system, not kept per process. They are set to zero once when xv6 boots and are never reset afterwards, not even by `throttleSyscall`. Every rate limited process adds to the same pair of counters.
- A call only reaches the counters if it is a call to the system call that is currently being rate limited. **Calls to anything else are not counted at all.**
- A process that has never called `throttleSyscall`, and whose parent had no limit, behaves exactly like unmodified xv6. In particular a newly created process must never pick up a limit left behind by an unrelated process that has already exited.

Mechanics:

- Update `syscall.c` and `struct proc`. A refused call must return -1 without the system call handler being run at all.
- *The per-process fields go in `struct proc` in `kernel/proc.h`. The three functions that create, copy and destroy a process are `allocproc()`, `fork()` and `freeproc()`, all in `kernel/proc.c`; that is where those fields have to be set up, inherited and cleared.*
- Use the global kernel tick counter `ticks`, defined in `kernel/trap.c` and already declared for you in `kernel/defs.h`. It is shared between all CPUs.

The two counters are shared as well, so give them a lock of their own and initialise it once at boot. The other init calls in `kernel/main.c` are a good place to do that. `kernel/sysproc.c` already includes `spinlock.h`.

- Define the counters and their lock as ordinary globals in `kernel/sysproc.c` next to your handlers, and declare them in `kernel/defs.h` so that `kernel/syscall.c` can reach them as well.

Required User Program:

`thr syscall_num min_ticks` , which installs the limit, calls getpid() three times printing each result, forks, has the child do the same, and finally prints the statistics.

The body of the program is given below.

```
int main(int argc, char *argv[])
{
    struct throttle_stats ts;
    int i, interval;

    if(argc != 3){
        fprintf(2, "Usage: %s syscall_num min_ticks\n", argv[0]);
        exit(1);
    }

    interval = atoi(argv[2]);
    if(throttleSyscall(atoi(argv[1]), interval) < 0){
        fprintf(2, "throttleSyscall failed\n");
        exit(1);
    }

    printf("-- parent burst --\n");
    for(i = 1; i <= 3; i++)
        printf("%d: getpid() = %d\n", i, getpid());

    if(fork() == 0){
        printf("-- child burst (inherits the limit) --\n");
        for(i = 1; i <= 3; i++)
            printf("%d: getpid() = %d\n", i, getpid());
        exit(0);
    }
    wait(0);

    if(getThrottleStats(&ts) < 0){
        fprintf(2, "getThrottleStats failed\n");
        exit(1);
    }

    printf("-- Throttle Stats (system wide) --\n");
    printf("allowed: %d, denied: %d\n", ts.allowed, ts.denied);
    exit(0);
}
```

See the sample I/O for clarification. The three commands are run one after another, in the same session:

```
$ thr 11 20
-- parent burst --
1: getpid() = 3
2: getpid() = -1
3: getpid() = -1
-- child burst (inherits the limit) --
1: getpid() = 4
2: getpid() = -1
3: getpid() = -1
-- Throttle Stats (system wide) --
allowed: 2, denied: 4
$ thr 11 20
-- parent burst --
1: getpid() = 5
2: getpid() = -1
3: getpid() = -1
-- child burst (inherits the limit) --
1: getpid() = 6
2: getpid() = -1
3: getpid() = -1
-- Throttle Stats (system wide) --
allowed: 4, denied: 8
$ thr 5 20
-- parent burst --
1: getpid() = 7
2: getpid() = 7
3: getpid() = 7
-- child burst (inherits the limit) --
1: getpid() = 8
2: getpid() = 8
3: getpid() = 8
-- Throttle Stats (system wide) --
allowed: 4, denied: 8
```

Explanation:

- 11 is the system call number of `getpid` and 5 is that of `read`.
- In the first command the parent is allowed one call to `getpid` and the next two arrive inside the same 20 tick window, so they are refused. The child inherits the limit but starts with a fresh clock, so its own first call is allowed and its next two are refused. Two calls got through in total and four were refused, which is exactly what the counters report.
- The second command runs the same program again. Because the counters are system wide and are never reset, they carry on from where the first command left them, reaching 4 and 8.
- The third command rate limits `read` instead of `getpid`. **This program never calls `read`, so no call is ever subject to the limit: every `getpid` succeeds in both processes and the counters stay exactly where the second command left them.**
- Each command is a new process, which is why the pids differ from one command to the next. The pids on your machine may differ; the pattern of values must not**.**

Submission:

```
git add --all
git diff HEAD > ../{studentID}.patch
```